RankeDB Applications: Chat, Memory Agents, and the Coordination Problem

Florian Noël
2026-04-09 — sketch — CC-BY-4.0

Abstract

TBD — this paper is a structured idea collection, not yet a coherent argument.

We explore the application layer built on the RankeDB provenance database: a chat interface backed by a memory agent that uses the semantic graph as source of truth, with multiple background agents contributing to context. The paper documents design questions, early experiments, and the coordination problem that emerges when multiple agents compete for conversational attention — a problem whose solution motivates future work on attention economics.

1. Introduction

- Papers 1 and 2 deliver a populated provenance database with a semantic graph
- This paper asks: what can you *do* with it?
- Primary application: conversational interface with provenance-grounded memory
- This is an idea sketch, not a finished architecture

1.1. Design Goals

The memory agent and the surrounding chat stack are designed to deliver the **five core long-term memory abilities** identified by Wu et al. (2025, *LongMemEval*, ICLR 2025, arXiv 2410.10813v2). We adopt their definitions verbatim as design goals for the application layer:

- **Information Extraction (IE).** *"Ability to recall specific information from extensive interactive histories, including the details mentioned by either the user or the assistant."*
  → **Agent requirement:** the memory agent must be able to retrieve specific facts from the semantic graph, including facts the *assistant* produced in past turns. Assistant utterances are first-class memory, not transient scaffolding — a restaurant the assistant recommended yesterday is a fact the user can ask about tomorrow. The agent must query with enough specificity to return the particular fact asked for, not a summary of the surrounding conversation.
- **Multi-Session Reasoning (MR).** *"Ability to synthesize information across multiple history sessions to answer complex questions that involve aggregation and comparison."*
  → **Agent requirement:** the memory agent must aggregate, count, and compare entities across session boundaries. *This is the ability on which the RankeDB stack is expected to have its clearest structural advantage* (see §7.3): because sessions are a container in the data model and not an organizing principle, cross-session synthesis is a direct L2 traversal rather than a reconstruction from summaries. The agent's job is to exploit that advantage by querying at the entity level, not at the session level.
- **Knowledge Updates (KU).** *"Ability to recognize the changes in the user's personal information and update the knowledge of the user dynamically over time."*
  → **Agent requirement:** when answering a question about a fact that has been superseded, the agent must select the current version, not an earlier version, without losing the ability to return prior versions when the question is about history. The append-only data model keeps both versions available; the agent's view-configuration policy decides which one answers a given question.

- **Temporal Reasoning (TR).** *"Awareness of the temporal aspects of user information, including both explicit time mentions and timestamp metadata in the interactions."*
  → **Agent requirement:** the agent must answer questions that require arithmetic on time — durations, orderings, most-recent selections — using the `valid_from`/`valid_until` on L2 edges and transaction time from the L1 DAG. Temporal filters must be a first-class query primitive, not a post-hoc filter applied to results.
- **Abstention (ABS).** *"Ability to identify questions seeking unknown information, i.e., information not mentioned by the user in the interaction history, and answer 'I don't know.'"*
  → **Agent requirement:** the agent must refuse to answer when no provenance chain supports a claim. This is the core of the grounded-responses discussion in §2.3: the agent's willingness to say "I don't know" depends on the provenance DAG being reachable, non-empty, and honest about its own reach. Abstention failures are hallucinations — they mean the agent manufactured evidence that does not exist in the database.

These five goals are the rubric we evaluate the chat stack against in §7 (LongMemEval). They also frame the design decisions throughout this paper: the memory agent's retrieval strategy, the background-agent coordination (§3, §4), and the reading strategy (§2.3) are all shaped by which of the five abilities each choice improves or undermines.

TODO — Reading for §1 (framing the application layer against PKG research and tools-for-thought):

This paper currently has no framing in the PKG academic community or the tools-for-thought lineage. Both are essential context — see PDF1 §4 ("Personal knowledge graphs are an active research field") and §8 ("Epistemology as infrastructure").

Personal Knowledge Graph research community (must engage):

- **Balog, K. (University of Stavanger). "Personal Knowledge Graphs: A Research Agenda."** ICTIR 2019. Foundational paper. researchgate.net/publication/367481375. **The academic PKG community has been building toward systems like RankeDB since 2019.** `read.pdf`. **Priority: H.**
- **"An Ecosystem for Personal Knowledge Graphs"** (ScienceDirect 2024, S2666651024000044). Defines PKGs around data ownership by a single individual and personalized service delivery. `read.pdf`. **Priority: M.**
- **PKG API (WWW Companion 2024).** ACM 3589335.3651247 + ACM 3341981.3344241. Proposes an RDF-based PKG vocabulary with provenance and access rights. `read.pdf`. **Priority: M.**
- **PDF1 key framing to lift:** *"The academic community primarily targets recommendation and personalization use cases — not the cognitive augmentation RankeDB pursues."* **This is RankeDB's position relative to Balog's line of work — state it explicitly in §1. Priority: H.**

Tools-for-thought lineage (intellectual ancestors):

- **Matuschak, A. & Nielsen, M. (2019). "How Can We Develop Transformative Tools for Thought?"** fluxent.com 2019-10-04. Catalyzed the tools-for-thought movement. Called for tools that *"change and expand the range of thoughts human beings can think."* `read.pdf`. **Priority: M.**
- **Engelbart, D. (1962). "Augmenting Human Intellect: A Conceptual Framework."** Foundational augmentation vision. `read.pdf`. **Priority: L.**
- **Bush, V. (1945). "As We May Think."** *Atlantic Monthly.* The Memex. Tracing roots. `read.pdf`. **Priority: L.**

- **PDF1 framing:** RankeDB implements the tools-for-thought vision *"through formal epistemological infrastructure"* rather than bidirectional linking (Obsidian/Roam). `read.pdf`. **Priority: M.**

2. The Memory Agent

2.1. Role

- The memory agent is the primary consumer of RankeDB
- It is a worker — reads all three levels via the API
- It is the human-facing interface to the knowledge graph
- It translates between natural language (chat) and structured knowledge (graph)

2.2. Retrieval Across Levels

- Entry via Level 2: associative search, entity traversal, semantic similarity
- Descent into Level 1: provenance chains, derivation history, competing interpretations
- Access to Level 0: original sources when needed for verification or full context
- The memory agent decides which level to query based on conversational need

TODO — Reading for §2 (memory agent landscape — note-taking tools vs semantic KGs):

PDF1 §4 has a devastating comparison of note-taking tools with graph views vs true knowledge graph systems. Lift this for §2 background.

- **Obsidian / Logseq.** Create link graphs — flat topologies of **untyped connections** between notes. **No entity resolution, no schema, no automated extraction.** Every link requires manual `[[bracketing]]` or `#tagging`. Obsidian's 2025 "Bases" feature adds database views but remains fundamentally a note-taking tool. `read.pdf`. **Priority: M.**
- **Tana supertags.** Closest existing tool to ontology-based approaches — nodes typed with supertags gain structured fields and computed values. But supertags must be **manually assigned**, no entity resolution, relationships aren't typed graph edges. `read.pdf`. **Priority: L.**
- **Mem.ai. Philosophical counterpart to the RankeDB-based chat stack** — shares "no manual organization" philosophy, AI organizes automatically. *Key contrast:* Mem treats AI as the organizer (opaque ML inside the product); the RankeDB stack separates concerns — the data structure is inspectable and LLM-free (Paper 1), workers that use LLMs operate above it (Paper 2), and the memory agent that translates natural language to graph queries operates above them (this paper). **Mem: "AI knows best." RankeDB stack: "the graph is the truth; inference happens in workers and agents above the database, never inside it."** `read.pdf`. **Priority: H — state this dichotomy explicitly in §2.**
- **Solid (Tim Berners-Lee).** Prioritizes data sovereignty through pods using W3C standards. opencommons.org/The_Solid_Protocol. **Leigh Dodds (2024): "Solid just isn't ready for general adoption"** — Pod API is essentially a document store without query capabilities. blog.ldodds.com/2024/03/12/baffled-by-solid. **Solid solves storage and access, not intelligence.** `read.pdf`. **Priority: M.**
- **PDF1 summary framing:** *"RankeDB leapfrogs all of these by building a true semantic knowledge graph with typed entities, typed relationships, automated extraction via worker DAGs, entity resolution with conviction scoring, and full provenance — while maintaining the personal-scale design and no-manual-tagging philosophy that tools-for-thought users expect."* **Use verbatim as §2 closing framing.**

2.3. Grounded Responses

- **Open question:** Should every claim in a response be traceable to a graph node?
- If yes: the agent operates like a RAG system where the graph is the corpus
- If no: the agent can reason freely but must distinguish graph-grounded from inferred
- **Idea:** Provenance annotations on responses — "I know this because [link to L1 Thought]"

TODO — Reading for §2.3 (grounded retrieval strategies — GraphRAG landscape):

PDF1 §5 "GraphRAG's deterministic retrieval" is the primary source. This is the bridge to Paper 4 — understand GraphRAG selection strategies to position RankeDB's Stacker ecology.

- **Microsoft GraphRAG** (Edge et al. April 2024). Entity/relation extraction + Leiden community detection + pre-built community summaries. User selects local/global/DRIFT mode; deterministic pipeline. medium.com/@zilliz_learn/graphrag-explained. `read.pdf`. **Priority: H.**
- **Neo4j GraphRAG package.** VectorRetriever, HybridRetriever, Text2CypherRetriever — user configures which to use. analyticsvidhya.com/blog/2024/11/graphrag-with-neo4j. `read.pdf`. **Priority: M.**
- **Neo4j ToolsRetriever (August 2025). Closest published parallel to Stacker's ecology.** Registers multiple retrievers as "tools" and uses an LLM to dynamically select which to invoke per query. **Key contrast: Neo4j delegates selection to an LLM; RankeDB Stacker removes the selector entirely and lets results compete.** neo4j.com/blog/developer/introducing-toolsretriever-graphrag-python-package. `read.pdf`. **Priority: H — the exact comparison point for Paper 4.**
- **LlamaIndex Property Graph Index.** LLMSynonymRetriever, VectorContextRetriever, CypherTemplateRetriever — concurrent retrievers but manual configuration. llamaindex.ai/blog/introducing-the-property-graph-index. `read.pdf`. **Priority: M.**
- **Adaptive RAG (Jeong et al. 2024).** Classifier-based complexity routing to different strategies. edenai.co/post/the-2025-guide-to-retrieval-augmented-generation-rag. `read.pdf`. **Priority: M.**
- **RAG-Fusion (Reciprocal Rank Fusion).** Combines multiple retrieval results via RRF. arxiv.org/html/2506.00054v1. *Closest to execution-competition but at ranking stage, not retrieval stage.* `read.pdf`. **Priority: M.**
- **DRIFT Search, LazyGraphRAG (Microsoft, 2024).** DRIFT combines global + local iterative refinement; LazyGraphRAG reduces indexing to 0.1% of full GraphRAG. `read.pdf`. **Priority: L.**
- **UaG — Uncertainty-Aware Graph (CIKM 2024).** Conformal prediction incorporated into KG-LLM reasoning; uses uncertainty to guide reasoning paths. *Closest academic work to "provenance as query direction" — directly relevant to grounded-response design.* `read.pdf`. **Priority: M.**

3. Background Agents

3.1. Proactive Researcher

- Runs continuously in the background
- Monitors conversation topics, searches RankeDB for related knowledge
- Pushes findings into conversational context
- **Open question:** visible or invisible to the user?
  - Visible: "I found something related…" — transparent but potentially noisy
  - Invisible: silently enriches context — cleaner UX but less trust
  - Hybrid: silent injection with option to inspect ("why did you know that?")

3.2. Memory Filler

- Extracts facts from ongoing chat and writes them back to RankeDB
- Chat → Record (L0) → extraction worker → Thoughts (L1) → entities (L2)
- The conversation feeds the graph, the graph feeds the conversation — feedback loop
- **Open question:** real-time or batched? Per-message or per-conversation?

3.3. Verification Agent

- When the memory agent cites a graph node, the verifier checks the provenance chain
- Can the claim actually be derived from the cited sources?
- Catches "fake citations" — agent hallucinating a provenance link
- **Open question:** how deep does verification go? L2→L1 sufficient? Or L2→L1→L0?

TODO — Reading for §3 (AI memory architecture for background agents):

PDF1 §7 and PDF2 §4 give you the current landscape of AI agent memory — which is the field Paper 3 enters.

- **Graphiti/Zep (arXiv 2501.13956, January 2025). Must read.** Bi-temporal KG for AI agent memory; same graph DBs as RankeDB (FalkorDB/Neo4j); 94.8% DMR benchmark, 300ms P95. Every entity/relationship traces to source "episodes." getzep.com 2025 report + neo4j.com/blog/developer/graphiti-knowledge-graph-memory. `read_2.pdf`. **Priority: H.**
- **Google Always-On Memory Agent (March 2026). Anti-RankeDB.** ConsolidateAgent every 30 min. LLM as truth arbiter. Read as the design to *not* converge toward. digit.in/features/general/googles-new-ai-agent-remembers-everything; elephaant.com/blog/google-always-on-memory-agent-vector-db-alternative-2026. `read.pdf`. **Priority: H.**
- **Amazon Bedrock AgentCore (2025).** Append-only memory patterns — marks outdated memories INVALID instead of deleting. aws.amazon.com/blogs/machine-learning/building-smarter-ai-agents-agentcore-long-term-memory-deep-dive. `read_2.pdf`. **Priority: M — partial alignment with RankeDB.**
- **Collaborative Memory (arXiv 2505.18279).** Each memory fragment carries immutable provenance attributes. `read.pdf`. **Priority: M.**
- **PROV-AGENT (Souza et al., IEEE e-Science 2025, arXiv 2508.02866).** First provenance framework for AI agent workflows; extends W3C PROV with agent-specific metadata. *Within traditional workflow orchestration, not provenance-first.* `read_2.pdf`. **Priority: M.**
- **ICLR 2026 Workshop on Memory for LLM-Based Agentic Systems (MemAgents).** Calls for research on *"provenance-aware retrieval"* and *"structured memory access control."* OpenReview U51WxL382H; arXiv 2603.10062. **Positions Paper 3 at the current research frontier.** `read_2.pdf`. **Priority: H — cite to place Paper 3 in 2026 context.**
- **"Graph-Native Cognitive Memory for AI Agents" (arXiv 2603.17244, 2025-2026).** Applies AGM belief revision to Neo4j AI memory. **Closest published work to RankeDB epistemology.** `read.pdf`. **Priority: H.**

4. The Coordination Problem

4.1. Multiple Agents, One Conversation

- Memory agent, researcher, verifier — all want to contribute
- Only one conversational turn at a time
- Who speaks? When? How much context can each inject?

4.2. Naive Solutions

- Round-robin: fair but dumb — irrelevant agents waste turns
- Priority queue: static ranking — inflexible, doesn't adapt to conversational dynamics
- Central orchestrator: single point of control — bottleneck, doesn't scale

4.3. The Attention Problem

- This is not a scheduling problem — it's an attention allocation problem
- Relevant context is abundant, conversational bandwidth is scarce
- Need a mechanism where agents *compete* for attention based on relevance
- **This is the bridge to Paper 4** — the coordination problem motivates attention economics

TODO — Reading for §4 (the coordination problem as bridge to Paper 4):

Full detail lives in Paper 4's reading queue. For Paper 3's §4, the key thing is to acknowledge what currently exists so the coordination problem has a clean motivation.

- **Neo4j ToolsRetriever (August 2025).** The closest published analog. LLM-mediated tool selection across multiple retrievers. **RankeDB removes the selector.** neo4j.com/blog/developer/introducing-toolsretriever-graphrag-python-package. `read.pdf`. **Priority: H.**
- **Adaptive RAG (Jeong et al. 2024).** Classifier-mediated routing. `read.pdf`. **Priority: M.**
- **RAG-Fusion with Reciprocal Rank Fusion.** Ensemble at the ranking stage. `read.pdf`. **Priority: M.**
- **PDF1 verdict to use as §4 closing:** *"RankeDB removes the selector entirely, letting results compete directly. This ecological competition model has no direct precedent in the published literature."* **Priority: H.**

5. Chat Engine Requirements

5.1. Context Management

- Finite context window must be allocated across: conversation history, agent injections, retrieved knowledge
- Context is a scarce resource — who gets how much?
- **Observation:** this infrastructure is shared with Paper 4's stacker system
- Building the chat engine for Paper 3 *is* building the substrate for Paper 4

5.2. MCP Integration

- Memory agent as MCP server: exposes RankeDB capabilities to any MCP-compatible chat client
- Tools: search graph, traverse provenance, retrieve source, add record
- Decouples memory from chat UI — any client can use RankeDB memory

5.3. Session Architecture

- Persistent vs. ephemeral sessions
- Conversation history as Records in RankeDB (feedback loop)
- Multi-device access to same memory

6. Open Questions

Collected during design, to be resolved through experimentation:

- How to prevent citation hallucination without making every response slow?
- How much background agent activity is useful vs. noisy?

- Should the user see the provenance graph or only the conversational surface?
- What's the right granularity for memory extraction from chat — per message, per topic, per session?
- How to handle contradictions between chat statements and existing graph knowledge?
- When multiple agents disagree, how does the user experience this?

7. Evaluation

7.1. Benchmark: LongMemEval

We evaluate the chat-assistant stack built on RankeDB against **LongMemEval** (Wu et al., ICLR 2025; arXiv 2410.10813v2), a benchmark for long-term memory in chat assistants. LongMemEval consists of 500 manually curated questions embedded within freely scalable user-assistant chat histories and tests five core memory abilities: **information extraction (IE)**, **multi-session reasoning (MR)**, **knowledge updates (KU)**, **temporal reasoning (TR)**, and **abstention (ABS)**. Two standard settings are provided: **LongMemEval_S** at approximately 115k tokens per problem, and **LongMemEval_M** at 500 sessions and around 1.5 million tokens per problem. Benchmark and code: `github.com/xiaowu0162/LongMemEval`.

> **Note.** The LongMemEval paper is a primary reference for this work, well beyond the evaluation section. Its unified view of memory-augmented chat assistants across **three execution stages** — indexing, retrieval, reading — and **four control points** — value, key, query, reading strategy — provides vocabulary and decomposition we adopt throughout Paper 3. Wu et al.'s §5.2–§5.5 report four experimental findings that directly inform the memory-agent design in §2 of this paper; we map each one to the RankeDB stack in §7.4 below.

RankeDB itself is a domain-agnostic database with no LLM and no notion of a chat assistant. Running LongMemEval therefore means evaluating the **full chat-assistant stack** built on top of RankeDB — ingest workers that load dialogue history as Records, extraction workers that produce Thoughts with provenance (Paper 2), and the memory agent (this paper) that answers benchmark questions by querying the API. The benchmark measures this entire stack. Pure database-level metrics (throughput, latency, storage, rebuild cost) are the subject of Paper 1 §7 and are not evaluated here.

7.2. Why LongMemEval alone is sufficient

LongMemEval strictly dominates every prior long-term memory benchmark on capability coverage, and matches or exceeds them on scale. The following comparison is adapted from Table 1 of the LongMemEval paper:

Benchmark	Domain	Sessions	Context depth	IE	MR	KU	TR	ABS	Covered
MSC (Xu et al., 2022a)	Open-domain	5k	1k	–	–	–	–	–	0/5
DuLeMon (Xu et al., 2022b)	Open-domain	30k	1k	–	–	–	–	–	0/5
MemoryBank (Zhong et al., 2024)	Personal	300	5k	✓	–	–	–	–	1/5
PerLTQA (Du et al., 2024)	Personal	4k	1M	✓	–	–	–	–	1/5

LoCoMo (Maharana et al., 2024)	Personal	–	10k	✓	✓*	–	✓	–	3/5
DialSim (Kim et al., 2024)	TV shows	1k–2k	350k	✓	✓	–	✓	–	3/5
LongMemEval (Wu et al., 2025)	Personal	50k	115k / 1.5M	✓	✓	✓	✓	✓	**5/5**

* *At most two sessions tested.*

No earlier benchmark covers more than three of the five core memory abilities, and LongMemEval matches or exceeds all of them on session count and context depth. A system evaluated on LongMemEval is evaluated on a strict superset of the capability axes tested by prior work. We therefore adopt LongMemEval as our single task benchmark without additional long-term memory benchmarks.

7.3. Mapping the five memory abilities to the RankeDB stack

Each of LongMemEval's five memory abilities corresponds to a specific part of the chat-assistant stack:

- **Information extraction (IE).** Executed by extraction workers (Paper 2 §3.4) that consume normalized dialogue Thoughts and produce entity/relation Thoughts in Level 1, which are projected into Level 2.
- **Multi-session reasoning (MR).** *This is the ability where we expect the RankeDB stack to have its clearest structural advantage.* Sessions are a container in the raw data, not an organizing principle in the data model: the L2 semantic graph holds entities and facts unified across session boundaries by entity resolution (Paper 2 §4), and every node traces through provenance back to the specific session and utterance it came from. The memory agent can count, aggregate, or reason over entities as a direct L2 traversal, without reconstructing anything from per-session summaries. We still *represent* sessions (as L0 Records and L1 boundary Thoughts) because questions about sessions are themselves valid questions and the session context matters for ranking and display — but we do not *privilege* them as the primary unit of retrieval. Cross-session synthesis is the default path, not a query pattern layered on top.
- **Knowledge updates (KU).** A direct consequence of the append-only data model (Paper 1 §3.2). Updated facts do not overwrite prior facts; both coexist with temporal validity windows, and the memory agent selects via view configuration (most recent, highest confidence, or explicit provenance policy).
- **Temporal reasoning (TR).** Supported by `valid_from` / `valid_until` on every L2 edge, plus transaction time provided by the L1 DAG — the "emergent bitemporality" property of the stack.
- **Abstention (ABS).** Enabled by the provenance DAG: the memory agent can refuse to answer when no provenance chain supports a claim, rather than fabricate. The stack supports this architecturally; the abstention policy itself is a memory-agent concern (§2.3).

7.4. LongMemEval design findings and their implications

Wu et al. §5.2–§5.5 report four experimental findings from their unified memory-design framework. Each has a direct analog in how the RankeDB stack is organized.

§5.2 — Round granularity beats sessions. LongMemEval finds that "round" (single-turn) granularity is optimal for storing and retrieving history; further compression into individual user facts loses information overall but improves multi-session reasoning. On RankeDB, this tradeoff disappears: the

append-only DAG retains *every* granularity simultaneously — the raw dialogue Record at L0, the normalization Thought, the per-round Thought, the per-session summary Thought, the extracted fact Thought, and the L2 projection. The memory agent selects the granularity appropriate to the question via view configuration. No single-granularity commitment is required.

The consequence we expect to show on LongMemEval is that the stack has a structural advantage on *multi-session reasoning* in particular. Wu et al.'s result — that compression into facts helps MR specifically — is exactly the regime RankeDB is built for: we carry the facts (in L2), the rounds (in L1), and the full dialogue (in L0) at the same time. Where a session-centric system has to pick, we do not.

§5.3 — Key expansion with extracted facts improves retrieval. LongMemEval reports that expanding memory keys with extracted user facts adds **+9.4% recall@k** and **+5.4% QA accuracy** over a flat memory-values-as-keys baseline. In RankeDB terms, this is what Level 2 already is: the semantic index is built from extracted facts projected from L1, with every L2 node and edge carrying a provenance reference back to the content in L1. Key expansion is the architectural default, not an optimization layer.

§5.4 — Time-aware indexing improves temporal reasoning. LongMemEval proposes an indexing and query expansion strategy that explicitly associates timestamps with facts, reporting **+6.8%–11.3% recall improvement** on temporal questions with strong-LLM query expansion. The RankeDB stack has this at the schema level: every L2 edge carries `valid_from` and `valid_until`, and the L1 DAG provides transaction time. The memory agent can narrow search ranges directly from the edge properties without a separate temporal index.

§5.5 — Reading strategy matters even with perfect recall. LongMemEval reports up to **+10 absolute points of QA accuracy** from Chain-of-Note (Yu et al., 2023) and structured data format (Yin et al., 2023) on top of the same retrieved items. This is a reader-level finding that motivates §2.3 of this paper: the memory agent should return structured entity-and-relation context (which RankeDB provides by default through the L2 projection) rather than raw dialogue text, and should use Chain-of-Note-style reading to trace provenance chains before answering.

7.5. What we report

For each of the five memory abilities, we intend to report:

- Accuracy on LongMemEval_S (115k tokens per problem) and LongMemEval_M (1.5M tokens per problem)
- Per-category breakdown (IE / MR / KU / TR / ABS)
- Comparison against the baselines reported by Wu et al. (2025): long-context LLMs, commercial chat assistants, and open-source memory systems
- Ablations that remove specific RankeDB properties:
  - No provenance descent (L2-only retrieval, no L1 derivation traversal) — expected to hurt MR and ABS
  - No temporal edges — expected to hurt TR
  - No append-only supersession (consolidated view only) — expected to hurt KU

8. Conclusion

- RankeDB + memory agent = a chat system with grounded, provenance-tracked memory
- Background agents create a richer but harder-to-coordinate system
- The coordination problem is real and motivates economic mechanisms (Paper 4)
- The chat engine infrastructure built here is the substrate for future stacker experiments

References

TBD — will reference Papers 1–2, agent memory literature, MCP specification, and Wu et al. (2025) "LongMemEval" (ICLR 2025, arXiv:2410.10813v2).

Sketch v0.1 — April 2026